Testing and Evaluation

Code Plagiarism Checker

SDN260S Software Design II · Group 8 · Topic 17

Masase N · 222794089

1. What was tested and why

The checker is only useful if people can trust what it says. If a renamed copy slips through, or two honest students get flagged, a lecturer would stop using it. So the testing asks four plain questions. Does it find copies that have been disguised? Does it leave unrelated work alone? Does it cope with awkward files without crashing? And is it fast and clear enough to use?

Thirteen tests were planned before anything was run. Three are normal cases (T-01 to T-03), six are edge cases (T-04 to T-09), and four look at processing time, error messages and how easy the command line and the window are to use (T-10 to T-13). The expected result for each was written down first, so the results could not be bent to fit.

2. How the testing was done

The tests were first run on 1 November 2026 on the final build, and then repeated on 8 November 2026 on the merged build, which includes the fixes made in between. Both builds have the same 18 classes and the same 87 unit tests. Each test used one of nine small sets of Java files made for the purpose (section 3). The program was driven in three ways: with file and folder paths on the command line, through the text menu with scripted input, and through the Swing window.

Item	Detail
Date of the runs	1 November 2026 (final build) and 8 November 2026 (merged build, all 13 tests repeated)
Computer	A Linux test machine with Java 21 and one processor
Tester and computer for sign-off	8 November 2026, merged build. T-01 to T-12: scripted run with AI assistance on the Linux test machine above. T-13 (the window): checked by hand by Ndugiselo Masase on a computer. On 1 November all of T-01 to T-13 were run by script, and a second full run the same day reproduced every score.
Program	Maven project, main class plagiarismchecker.Main
Flag threshold	70% (the default in SimilarityChecker)
Times	The program now prints its own time for the compare step (cleaning, tokenising and comparing). The 1 November whole-program times include about 60 ms of Java start-up; comparison-only times were measured separately.

Every scripted run was saved as a plain text transcript, which is listed with its command in the evidence folder and on the Evidence sheet of the test log. The 1 November checking run is saved as final_verification_run.txt, and the 8 November transcripts are in evidence_8Nov. A teammate’s Check-In 3 screenshots add independent evidence from a Windows computer: the Maven build, the Maven tests, the text menu, the sample folder and the wrong-path error.

3. The test datasets

Each dataset is a small folder of invented student programs with a known relationship between the files. None of it is real student work.

Folder	What is in it
T01	Two unrelated programs: average of temperatures and a palindrome checker
T02	A bubble sort and a copy with variables renamed, one comment changed and one extra print line
T03	Five programs: the bubble-sort pair and three unrelated ones (palindrome, bank account, Fibonacci)
T04	A valid file, an empty file (0 bytes) and a file with only blank space
T05	Two valid files, plus notes.txt, results.csv and a Java file with the extension .jav
T06	Two byte-for-byte identical files
T07	Two different one-line classes and a file that holds only comments
T08	A generated file of about 1,300 lines (100 methods) and a copy with 10 methods rewritten
T09	The same code twice with different spacing, brace style and comments

The project also ships eleven mock submissions in samples/mock, built with known relationships (renamed copies, a reordered copy, a partial copy, independent solutions and unrelated programs). They are used by the unit tests and as a second check on the threshold.

4. Results of the 13 tests

ID	Test	Expected	What happened (8 Nov)	Result
T-01	Two different programs	Low, not flagged	20.0%, not flagged	Pass
T-02	Two similar programs	High, flagged	91.3%, flagged	Pass
T-03	Multiple submissions	All pairs, sorted	10 pairs from 91.3% down to 15.6%; only A and B flagged	Pass
T-04	Empty file	Clear message, no crash	Both files rejected with “File is empty”; no crash	Pass
T-05	Invalid file type	Readable error	Three files rejected with “Only .java files can be checked”; valid pair compared at 20.0%. A folder scan now reports the three skipped files.	Pass
T-06	Duplicate file	100%, flagged	100.0%, flagged	Pass
T-07	Very small file	No crash, no false 100%	8.3%, 0.0%, 0.0%; no crash	Pass
T-08	Large file	Completes, time recorded	86.2%, flagged; about 12 ms to compare on 1 Nov, and 174 ms as the program’s own figure for the whole compare step on 8 Nov	Pass
T-09	Different formatting	High score	100.0%, flagged	Pass
T-10	Processing time	Recorded	153 to 270 ms whole program on 1 Nov. Since 8 Nov the program prints its own time: 11 to 174 ms on the datasets	Pass
T-11	Error messages	Readable and consistent	Consistent on the command line. In the text menu the reason now shows on screen, and a path with an illegal character gives a readable error instead of a crash	Pass (Fail on 1 Nov)
T-12	Text menu usability	Checked	8 of 8 checklist items Yes (7 of 8 on 1 Nov)	Pass (Fail on 1 Nov)
T-13	Window usability	Checked	8 of 8 checklist items Yes (checked by hand on a computer)	Pass

All thirteen tests passed on the merged build. On 1 November eleven passed and two failed (T-11 and T-12); both now pass after the fixes to DEF-10 and DEF-11 (section 7). The headline result is that the checker does what it was built to do. The renamed copy in T-02 scored 91.3%, and the same code with different spacing and comments in T-09 scored 100%, so the cleaning and token steps work. The unrelated programs in T-03 never rose above 29.0%. Every score on 8 November matched 1 November.

Two things deserve a closer look. Only_Comments.java in T-07 scores 0.0% against every file and the program still does not say that it had nothing to compare (DEF-13, a known limitation). And in T-05, a whole-folder scan used to skip notes.txt, results.csv and the .jav file without any message, so a misspelt extension simply disappeared. That is fixed: each skipped file is now reported.

5. Checking the 70% threshold

A good threshold sits above the highest score between unrelated programs and at or below the lowest score between clearly similar ones. Across the 14 pairs in the datasets, the highest unrelated score was 29.03% and the lowest similar score was 91.30%, a gap of 62.27 points. A threshold of 70% sits comfortably inside it, and every one of the 14 pairs landed on the correct side.

The mock submissions are a harder test, because they include in-between cases:

Kind of pair (mock data)	Score	At 70%
Renamed copies (Alice and Bob, Gina and Hank)	100.0%	Flagged
Renamed and reordered copy (Dan and Eve)	80.4%	Flagged
Partial copy (Gina and Kim)	61.0%	Not flagged
Independent solutions to the same task	about 29% to 51%	Not flagged
Unrelated programs	about 16% to 45%	Not flagged

At 70%, exactly the three built-in copies are flagged. The partial copy at 61.0% is not, and it is only 9 points above the highest independent solution at 51.1%. That is the trade-off. Lowering the threshold to catch partial copies would start to flag honest work. The first prototype showed the opposite problem: its renamed copy scored 52.27%, below even the old 60% placeholder. The group approved the 70% value on 1 November 2026.

6. Performance

For the small datasets the comparison itself takes well under a millisecond, and the whole program, including starting Java, finishes in 150 to 270 ms. Since the 8 November build the program prints its own time for the compare step, so these figures no longer have to be taken with an outside timer.

Dataset	Files	Tokens in total	Comparison only (ms)	Whole program (ms)
T-07 very small	3	17	0.1	153
T-09 formatting	2	302	0.2	168
T-06 duplicate	2	234	0.3	207
T-01 different	2	188	0.8	231
T-02 similar	2	311	0.8	222
T-03 five files	5	659	5.6	239
T-08 large pair	2	14,664	12.0	254

The project’s Benchmark class compares generated files and shows how the time grows with the number of files. Work grows with the number of pairs, which rises with the square of the number of files, and the figures follow that:

Files	Pairs	1 thread (ms)	2 threads (ms)
10	45	54.1	39.9
20	190	79.4	76.8
40	780	350.7	344.3

Files	Pairs	1 thread (ms)	2 threads (ms)
80	3,160	1,439.8	1,439.3
160	12,720	4,990.1	5,372.6

Two threads gave no real speed-up here because the test machine has only one processor, so the threads simply took turns. Timings also vary from run to run: the second full run on 1 November gave slightly higher figures (for example 17.6 ms for the large pair instead of 12.0 ms), so they are best read as rough. A computer with several cores should show a clear gain, and the parallel unit tests confirm that two threads give exactly the same results as one. This benchmark should be repeated on a real laptop before the numbers are quoted as a performance claim.

7. Problems found

The first dry run on 22 October, on the early prototype, found nine problems. They went to the owners and were re-tested on the final build. Eight were fixed by 1 November: every pair is now compared and sorted, there is a flag, empty files are rejected, comments and formatting are removed before comparing, a renamed copy scores high, error messages differ by cause, letters at the menu no longer crash it, and the window now works. The ninth, that the program did not print its own run time (DEF-07), is fixed in the 8 November build.

The final build then turned up four new problems. All four were re-tested on 8 November:

ID	Problem	Severity	Status on 8 Nov
DEF-10	In the text menu and the window, the reason a file was rejected went only to the log file. The screen said “0 file(s) loaded in total” or “No usable .java files were found”.	Medium	Fixed. The menu now prints the reason for each rejected file.
DEF-11	A path containing a character the system does not allow, typed at menu option 1, ended the program with a Java stack trace. A path pasted with quotation marks, as Windows does, should have done the same.	High	Fixed. The menu shows “Invalid path” and carries on. Quoted and padded paths now load.
DEF-12	Scanning a folder skipped files that are not .java without saying so.	Low	Fixed. Each skipped file is now reported.
DEF-13	A file with only comments and imports scores 0.0% against everything and nothing says it had nothing to compare.	Low	Known limitation. Not fixed in the 8 Nov build; the program behaves as before.

DEF-10 and DEF-11 were the reason T-11 and T-12 were marked as failed on 1 November. Both were re-tested on 8 November and the two tests now pass. The window’s part of DEF-10 and DEF-07 (a dialog listing skipped files, the time in the status line) was read from the code, and the window was then checked by hand for T-13.

8. The unit tests and other evidence

The project has 87 unit tests in 17 test classes, and all of them pass. This was confirmed in three places: a Maven run on a group member’s computer on 23 October, a clean build and test run in a teammate’s NetBeans on 31 October, and a JUnit run in the test environment on 1 November. The merged build of 8 November holds the same 87 tests. They were not run again on the Linux test machine that day, because it has no Maven. A separate small test harness for a different, Longest Common Subsequence calculator also passes its eight checks, but it belongs to a different set of classes and is not counted among these results.

9. Limits of this testing

- T-01 to T-12 were scripted and run on one test machine, and T-13 was checked by hand on one computer. Repeating them on more computers would confirm the timings in particular.
- The window was checked with a script on a virtual screen on 1 November and by hand on 8 November. Each check covered the eight items on the usability checklist.
- All the test data is invented and small. The threshold has not been tried on real student submissions.
- The checker only reads Java, and it measures structure, not behaviour. Two programs that work the same way but are built differently can score low.
- A file that holds only comments and imports still scores 0.0% without a message (DEF-13).
- The timings come from a machine with one processor, so the speed-up from extra threads has not been measured.

10. Conclusion

The checker finds disguised copies, leaves unrelated programs alone and survives awkward files, which were the three things most worth testing. The weak spots found on 1 November were mostly about communication: the reasons a file was rejected were not always shown on screen, and one kind of typed path could stop the text menu. Both are fixed, along with the silent skipping of files in a folder scan and the missing run time, and all 13 tests now pass. The one open item is DEF-13: a file with only comments scores 0.0% without a warning. None of the problems ever affected the scores themselves.

Appendix: repeating the tests

From the project folder, after compiling. On Windows use target\classes and backslashes.

```
java -cp target/classes plagiarismchecker.Main
  test_datasets/T02_similar_programs/Student_A.java
  test_datasets/T02_similar_programs/Student_B.java
```

```
java -cp target/classes plagiarismchecker.Main
  test_datasets/T03_multiple_submissions
```

```
java -cp target/classes plagiarismchecker.Main
  test_datasets/T04_empty_file/Student_A.java
  test_datasets/T04_empty_file/Empty.java
```

```
java -cp target/classes plagiarismchecker.Main --cli
```

```
java -cp target/classes plagiarismchecker.Benchmark
```

```
mvn clean test
```

Enter each result on the Test Log sheet, and use the Evidence sheet to see which transcript belongs to which test.

Addendum: final build of 8 November 2026 (evening)

This page was added after the last run above. Where it differs from sections 4 to 10, this page applies.

1. DEF-13 is fixed.

A file with only comments and imports still scores 0.0%, but the program now says so: the text menu prints "[WARN] No code to compare in Only_Comments.java (only comments or blank lines). It will score 0.0%.", the window shows a warning dialog, and the log records a WARN line. This was re-tested on the final build by a scripted run on the command line and in the text menu (transcripts in docs/evidence_8Nov_final/). The window dialog was not clicked through. DEF-07, DEF-10, DEF-11, DEF-12 and DEF-13 are now all fixed, so no defect remains open, and the references to DEF-13 as a known limitation in sections 4, 7, 9 and 10 no longer apply.

2. Maven tests on the merged build.

Bandile Qwane ran mvn test on the merged build on a Windows computer on 8 November 2026 at 18:36: 87 tests run, 0 failures, 0 errors, BUILD SUCCESS. The screenshots are kept with the group's testing evidence. This replaces the remark in section 8 that the merged build was not run again with Maven.

3. Real benchmark.

The benchmark in section 6 was run on a one-processor machine and is superseded by the measured run in docs/REAL_EXECUTION_TIME_BENCHMARK.md (Kagiso Lengolo, 8 November 2026, OpenJDK 21 on a 2-processor machine, median of 5 runs after 2 warm-ups). For 80 files (3,160 pairs) the comparison took 904 ms on one thread and 575 ms on two, about 1.6 times faster. Going from 20 to 40 files raised the time from 61 ms to 242 ms, about four times, as the $O(n^2)$ analysis predicts. These figures come from a test machine with few cores, not a student laptop.

4. Final test log.

The Test Log workbook in docs/ holds the 1 November run (11 passed, 2 failed) and the 8 November repeat (13 of 13 passed), with DEF-13 updated to Fixed.